

DAT120 og DAT125 øvingsoppgave 2

Kontrollstrukturer

Læringsmål

Du skal lære å bruke kontrollstrukturer som if, for og while. Du skal lære å formulere enkle algoritmer som bruker disse kontrollstrukturene.

Kunstig Intelligens

Denne øvingen er ment å løses uten bruk av KI. Du bør prøve å løse alle oppgavene uten bruk av KI. Å stå fast litt under løsning av programmeringsoppgaver er helt normalt, men hvis ikke kommer videre så er det OK å be KI om hjelp. Se dokumentet «studentguide til KI-bruk» for hvordan bruke KI på måter som hjelper heller enn hindrer læring. Hvis du bare ber KI om å løse øvingen for deg har du ikke lært det du skulle fra øvingen.

Basis Deloppgaver

Du velger selv om du løser deloppgavene her eller de under «deloppgaver for noen studieretninger» for din studieretning hvis din studieretning er med her.

- a) **Repetisjon:** Skriv et Python-script som regner ut drivstoffprisen for en kjøretur. Du velger selv når du skriver dette scriptet hva slags drivstoff bilen bruker. Programmet skal ta inn:
 - a. Pris pr. enhet drivstoff (liter bensin / liter diesel / kilowatt-time for elbil)
 - b. Hvor langt bilen kjører på en liter (evt. Kilowatt-time)
 - c. Lengde på kjøreturen.
- b) **If-setninger:** Et script «elbil_lading_startkode.py» er lagt ved øvingen. Dette scriptet leser inn et antall kilometer og rekkevidden til el-bilen og regner ut hvor mange ladinger som trengs underveis for å kjøre denne distansen med en el-bil gitt at den starter fulladet. Bruk dette scriptet som basis for å løse følgende oppgaver:
 - a. Modifiser scriptet slik at det skriver ut at du ikke trenger å lade hvis distansen er mindre enn 80% av maksdistanse. Er distansen mer enn 80% så bør du lade minst en gang. Du lader på 20% for å ha litt margin.
 - b. Modifiser scriptet slik at du antar at du i hver lading starter å lade fra 20% og lader til 80%. Du lader til 80% heller enn 100% siden ladingen går mye tregere de siste 20 prosentene og det er ofte køer på ladestasjonene. Anta at du starter kjøreturen fulladet.
- c) **While-løkke:** Det er delt ut et program som er en start på følgende spill: Datamaskina lager et tilfeldig tall mellom 1 og 1000. Så skal spilleren gjette tallet. For hver omgang skriver programmet ut om spilleren gjettet for lavt, riktig eller for høyt. Målet er å gjette riktig med å få forsøk som mulig. Programmet som er delt ut inneholder en del feil samt at et fint kan forbedres:
 - a. Programmet teller feil, det rapporterer alltid 0 forsøk!
 - b. Programmet skal avslutte rett etter at det har skrevet ut at brukeren har gjettet riktig

- c. Legg inn en mulighet for at spilleren legger inn hva som skal være høyeste tall. Så heller enn 1-1000 så skal programmet lage tall mellom 1 og det brukeren velger.
 - d. **Avansert:** For hver omgang skriv også ut om spillerens siste gjett var nærmere eller lengre unna det nest siste gjettet.
- d) **For-løkke:**
- a. Enkel: Skriv ut alle tallene fra og med 1 til og med 12 på hver sin linje.
 - b. Avansert: Lag et program som konverterer en serie med beløp fra en valuta til en annen. Programmet skal starte med å lese inn navnene på de to valutaene, kursen mellom dem, og hvor mange beløp som skal konverteres. Så skal programmet gå i ei for-løkke hvor det leser inn en verdi i den ene valutaen, konvertere den til den andre valutaen og skrive dette ut.
- e) **Turtle Graphics:** Lag et script som bruker Turtle Graphics til å tegne ut en mangekant. Les inn antall kanter fra brukeren, som må være minst 3. Er det lavere enn 3 skal scriptet skrive ut en feilmelding og avslutte. Ellers skal det tegne ut mangekanten gjennom å for hver kant tegne ei linje og så snu en vinkel lik 360/antall sider.
- f) **Nøstet løkke:** Start med å skrive et Python script som tegner fire sirkler som oppgitt i figur 1 under. Man kan gjøre dette ved å rotere skilpadda 90 grader mellom hver sirkel. Skriv deretter et script som tegner fire sirkler på denne måten flere ganger med større og større sirkler hver gang. Roter litt, for eksempel 10 grader, mellom hver gang. Eksperimenter med mønstrene du får med å bruke ulike rotasjonsvinkler og ulike antall sirkler av ulik størrelse. Du kan for eksempel få mønsteret oppgitt i figur 2 under.

Deloppgaver til noen studieretninger

Dette er deloppgaver som er ment å illustrere hvordan programmering kan brukes i ulike studieretninger. Disse er lagd av KI, men kvalitetssjekket av meg og folk på de ulike studieretningene. Du kan gjøre disse i stedet for eller i tillegg til de som er i kapittelet «Oppgaver». Merk at deloppgave b og c fra forrige avsnitt gir deg trening i å jobbe med en eksisterende kode, noe ingen av disse oppgavene gir. Dere som er på andre studieretninger enn de under bør gjøre oppgavene i kapitlet «Oppgaver» i stedet.

Bygg

Oppgave 1 – If-setninger: Lastkontroll for bjelke

Tema: Enkelt beslutningsgrunnlag ved dimensjonering.

Beskrivelse:

Skriv et program som:

1. Ber brukeren om å skrive inn **tillatt last** for en bjelke (i kN - kiloNewton).
2. Ber om **beregnet last** på bjelken (i kN).
3. Skriver ut om bjelken er **innenfor grense** eller **overbelastet**.
4. Hvis overbelastet, skriv også hvor mye den overskrider grensen (i kN).

Eksempelkjøring:

Tillatt last (kN): 45

Beregnet last (kN): 52

Bjelken er OVERBELASTET med 7.0 kN

Hint: Bruk if og enkel subtraksjon.

Oppgave 2 – For-løkke: Akkumulert materialkostnad per etasje

Tema: Kostnadsoverslag for råbygg.

Beskrivelse:

Anta at et bygg har **N etasjer**, og at materialkostnad per etasje er tilnærmet lik, men med et lite tillegg for økende kompleksitet for høyere etasjer.

Skriv et program som:

1. Ber brukeren oppgi **antall etasjer** (heltall).
2. Ber om **grunnkostnad per etasje** (i kr).
3. Ber om **tillegg per etasje** (i kr) som legges på *for hver etasje oppover* (f.eks. kr 500 per etasje).
4. Bruk en for-løkke til å **beregne og skrive ut kostnaden for hver etasje**, samt **total kostnad**.

Forutsetning: Kostnad for etasje i (start på 1) = grunnkostnad + $(i-1)$ *tillegg.

Eksempelkjøring:

Antall etasjer: 3

Grunnkostnad per etasje (kr): 200000

Tillegg per etasje (kr): 5000

Etasje 1: 200000 kr

Etasje 2: 205000 kr

Etasje 3: 210000 kr

Total kostnad: 615000 kr

Hint: Bruk en løpende sum-variabel.

Oppgave 3 – If + While: Klassifisering av lastkombinasjoner

Tema: Enkle kombinasjoner med klassifisering.

Beskrivelse:

Du skal beregne en serie med laster:

1. Be om **egenlast** (G, kN) og **nyttelast** (Q, kN).
2. Hvis minst en av disse er negative, avslutt.

3. Beregne **designlast** $S=1.35 \cdot G+1.5 \cdot Q$.
4. Hvis **S** er over en gitt **kapasitet** (oppgis i starten), print "**UAKSEPTABEL**", ellers "**OK**".

Eksempelkjøring:

Bjelkekapasitet (kN): 120

G (kN) for lasttilfelle 1: 60

Q (kN) for lasttilfelle 1: 30

Designlast: 121.5 kN -> UAKSEPTABEL

G (kN) for lasttilfelle 2: 50

Q (kN) for lasttilfelle 2: 20

Designlast: 112.5 kN -> OK

G (kN) for lasttilfelle 3: -1

Avslutter

Digital Serviceledelse

Oppgave 1 – If-setninger: Kundetilfredshet

En digital tjeneste lar brukerne gi en vurdering fra 1 til 10.

Lag et program som:

1. Spør brukeren om en vurdering (1–10).
2. Bruk if, elif, else for å skrive ut:
 - "Kunden er misfornøyd." hvis vurderingen er 1–4
 - "Kunden er nøytral." hvis vurderingen er 5–7
 - "Kunden er fornøyd!" hvis vurderingen er 8–10
 - "Ugyldig vurdering." hvis tallet er utenfor 1–10

Dette simulerer et enkelt kvalitetsmålingssystem.

Oppgave 2 – For-løkke: Antall brukere per time

Du skal lage et lite verktøy som viser hvor mange brukere som besøker en tjeneste hver time.

Lag et program som:

1. Spør om antall brukere som besøkte tjenesten per time
2. Bruk en **for-løkke** som går gjennom timene 1 til 6.

3. For hver time skal programmet lese inn antall brukere og skriver ut totalt antall brukere så langt.
4. Til slutt skal programmet skrive ut *totalen* for alle timene.

Hint: Du kan bruke en enkel variabel $total = total + brukere$.

Eksempelkjøring

Antall brukere time 1: 6

Totalt antall brukere så langt: 6

Antall brukere time 2: 2

Totalt antall brukere så langt: 8

Antall brukere time 3: 5

Totalt antall brukere så langt: 13

Antall brukere time 4: 3

Totalt antall brukere så langt: 16

Antall brukere time 5: 4

Totalt antall brukere så langt: 20

Antall brukere time 6: 5

Totalt antall brukere så langt: 25

Totalt antall brukere for alle timene: 25

Oppgave 3 – While-løkke: Ventetid i kundeservice

Et kundeservicesystem registrerer antall saker i kø.

For hver runde behandles **3 saker**, og du skal simulere dette.

Lag et program som:

1. Spør om hvor mange saker som står i kø.
2. Bruk en **while-løkke** som fortsetter så lenge antall saker er større enn 0.
3. I hver runde:
 - Fjern 3 saker ($saker = saker - 3$)
 - Skriv ut hvor mange saker som er igjen.
4. Når køen er tom eller negativ, skriv:
5. Alle saker er behandlet!

Dette simulerer en enkel prosessflyt for kundeservice.

Oppgave 4 – Nøstete løkker: Enkel rapport over responstid

Du jobber med å analysere responstider for en digital tjeneste.

Tjenesten måles på forskjellige tidspunkt og på forskjellige serverlokasjoner.

Vi har:

- Tidspunkter: kl 08, 12, 16
- Serverlokasjoner: Norge, Sverige, Danmark
- Responstid regnes som: $\text{responstid} = \text{baseverdi} + (\text{tillegg for land}) + (\text{tillegg for tidspunkt})$

For enkelhets skyld skal du bruke disse faste verdiene:

- Baseverdi = 50 ms
- Tillegg for land:
 - Norge = 0
 - Sverige = 10
 - Danmark = 20
- Tillegg for tidspunkt:
 - 08 = 0
 - 12 = 20
 - 16 = 40

Lag et program som:

1. Bruker en **ytre for-løkke** over tidspunkt (08, 12, 16).
2. Bruker en **indre for-løkke** over land (Norge, Sverige, Danmark). Bruk 0 for Norge, 1 for Sverige og 2 for Danmark. Dere kan komme tilbake til denne oppgaven når dere har hatt temaet «lister og plotting» og kjøre den direkte på ei liste av land i stedet.
3. Beregner og skriver ut responstiden for hver kombinasjon.

Eksempel (forkortet):

Tidspunkt: 08

0 -> 50 ms

1 -> 60 ms

2 -> 70 ms

Tidspunkt: 12

0 -> 70 ms

1 -> 80 ms

2 -> 90 ms

Elektro

Oppgave 1 – If-setning

Spenningsovervåking i en krets

Et måleinstrument rapporterer en spenning, og du skal sjekke om verdien ligger innenfor et trygt driftsområde.

Lag et program som:

1. Ber brukeren skrive inn **målt spenning (V)**.
2. Ber brukeren skrive inn **tillatt maksimal spenning (V)**.
3. Skriver ut
 - “OK – innenfor grensen” hvis spenningen er under eller lik maks
 - “ADVARSEL – overspenning!” hvis spenningen overstiger grensen
4. Ved overspenning skal programmet også skrive hvor mye grensen er brutt med.

Eksempel:

Målt spenning (V): 13.2

Maks tillatt (V): 12

ADVARSEL – overspenning!

Overskridelse: 1.2 V

Oppgave 2 – For-løkke

Strømforbruk per time i en krets

Du skal beregne det totale energiforbruket til et system som trekker **varierende strøm hver time**.

Siden studenten ennå ikke har hatt lister, leses hver time inn enkeltvis.

Lag et program som:

1. Ber brukeren oppgi **antall timer**.
2. I en for-løkke ber du brukeren skrive inn **strøm (Ampere)** for hver time.
3. For enkelhets skyld antar vi at spenningen er konstant **230 V**.
4. For hver time beregnes energien: $E = V * I * 1 \text{ time}$ (i watt-timer)
5. Skriv ut energi per time og totalt energi-forbruk.

Eksempel:

Antall timer: 3

Strøm time 1 (A): 2

Strøm time 2 (A): 3.5

Strøm time 3 (A): 1

Energiforbruk time 1: 460 Wh

Energiforbruk time 2: 805 Wh

Energiforbruk time 3: 230 Wh

Totalt energiforbruk: 1495 Wh

Oppgave 3 - While + If

Automatisk sikring som løser ut ved overstrøm

Du skal simulere en sikring som løser ut hvis strømmen blir for høy.

Lag et program som:

1. Ber brukeren skrive inn sikringsgrensen i ampere.
2. Starter en while-løkke der brukeren gjentatte ganger taster inn målt strøm.
3. Hvis strømmen overstiger grensen → skriv "Sikring utløst!" og stopp løkken.
4. Hvis ikke → skriv "OK", og fortsett å spørre om ny strømverdi.

Energi

Oppgave 1 – If-setning

Trykksjekk i et brønnsegment

Når et nytt brønnsegment bores, må man overvåke trykket for å sikre at det ligger innenfor et trygt område.

Lag et program som:

1. Ber brukeren skrive inn:
 - målt trykk (i bar)
 - maksimal tillatt brønntrykksgrænse (i bar)
2. Programmet skal skrive:
 - "Trykket er innenfor sikre grenser" hvis målt trykk $\leq$ maks
 - "ADVARSEL: Overtrykk!" hvis målt trykk $>$ maks
3. Hvis overtrykksituasjon: skriv også hvor mye grensen er overskredet.

Eksempel:

Målt trykk (bar): 190

Maks trykk (bar): 180

ADVARSEL: Overtrykk!

Overskridelse: 10 bar

Oppgave 2 – For-løkke

Beregning av total energiproduksjon over flere dager

En liten produksjonsbrønn leverer en viss energimengde per dag.

Lag et program som:

1. Ber brukeren skrive inn antall dager.
2. For hver dag bruker for-løkke til å be om **energi produsert den dagen** (i MWh).
3. Programmet skal skrive ut energien for hver dag og til slutt **total energiproduksjon**.

Eksempel:

Antall dager: 3

Produksjon dag 1 (MWh): 12

Produksjon dag 2 (MWh): 15.5

Produksjon dag 3 (MWh): 14

Total energiproduksjon: 41.5 MWh

Oppgave 3 – While-løkke

Simulering av fylling av en oljetank

En lagringstank fylles av en pumpe som leverer et konstant volum per minutt.

Lag et program som:

1. Ber om:
 - tankvolum (liter)
 - startvolum (liter)
2. Bruk en while-løkke som
 - Leser inn målt volum som ble pumpet inn i tanken dette minuttet
 - Stopper når tanken er full eller overfylt.
3. Skriv ut hvor mange minutter som gikk.

Eksempel:

Skriv inn volum for tanken i liter: 5000

Skriv inn startvolum for tanken i liter: 3200

Målt olje i liter dette minuttet: 150

Målt olje i liter dette minuttet: 120

Målt olje i liter dette minuttet: 90

Målt olje i liter dette minuttet: 130

Målt olje i liter dette minuttet: 150

Målt olje i liter dette minuttet: 150

Målt olje i liter dette minuttet: 150

Målt olje i liter dette minuttet: 120

Målt olje i liter dette minuttet: 90

Målt olje i liter dette minuttet: 80

Målt olje i liter dette minuttet: 90

Målt olje i liter dette minuttet: 110

Målt olje i liter dette minuttet: 150

Målt olje i liter dette minuttet: 150

Målt olje i liter dette minuttet: 150

Tanken er full etter 15 minutter

Oppgave 4 – Nøstet løkke

Temperaturraster for et oljerør (teksttabell)

Langs et oljerør blir temperatur målt flere steder. Du skal lage en *tekstlig temperaturtabell* som illustrerer målepunkter.

Lag et program som:

1. Ber brukeren angi:
 - antall rader (målepunkter langs rørets lengde)
 - antall kolonner (målepunkter rundt rørveggen)
2. Bruk en **nøstet for-løkke**:
 - Ytre løkke: rader
 - Indre løkke: kolonner
3. Innenfor hver kolonne, skriv ut et symbol:
 - H hvis målepunktet er "hot" (temperatur > grense)
 - C hvis målepunktet er "cold"

4. Brukeren skal taste inn temperaturen for hvert punkt når løkken går og legg resultatet (H eller C) inn i en streng. Legg et linjeskift til strengen for hver rad. Skriv ut strengen til slutt. Merk at i et reelt eksempel ville dataene kommet fra en fil eller direkte fra sensorene, men det vil kreve mer avansert programmering enn du kan på dette tidspunktet i DAT120.

Eksempelkjøring:

Antall rader: 2

Antall kolonner: 3

Temperatur for punkt (rad 1, kol 1): 70

Temperatur for punkt (rad 1, kol 2): 45

Temperatur for punkt (rad 1, kol 3): 80

Temperatur for punkt (rad 2, kol 1): 30

Temperatur for punkt (rad 2, kol 2): 55

Temperatur for punkt (rad 2, kol 3): 60

H C H

C H H

(Medium temp-grense kan du sette til f.eks. 50°C som en fast variabel uten input.)

Maskin

Oppgave 1 – If-setninger (temperaturkontroll i et teknisk system)

Et kjølesystem skal holde en metallkomponent mellom **10°C og 40°C**.

Skriv et program som:

1. Ber brukeren skrive inn temperaturen på komponenten.
2. Bruk if, elif og else for å skrive ut én av følgende meldinger:
 - "Temperaturen er for lav – aktiver oppvarming!" hvis temperaturen er under 10°C
 - "Temperaturen er innenfor trygg område." hvis den er mellom 10°C og 40°C
 - "ADVARSEL! Temperaturen er for høy – aktiver nedkjøling!" hvis den er over 40°C.

Tips: Du trenger bare enkle sammenligninger som <, <= og >=.

Oppgave 2 – For-løkke (beregne materialutvidelse)

Når en metallstang varmes opp, utvider den seg. Den lineære utvidelsen kan beregnes som:

$$\Delta L = \alpha \cdot L_0 \cdot \Delta T$$

der

- α er varmeutvidelseskoeffisienten
- L_0 er stangens opprinnelige lengde
- ΔT er temperaturendringen

Lag et program som:

1. Tar inn opprinnelig lengde L_0 fra brukeren.
2. Tar inn varmeutvidelseskoeffisienten α . For eksempel for stål er denne rundt $10^{-5} = 0.00001$.
3. Bruk en **for-løkke** til å beregne den nye lengden for temperaturøkninger fra **0°C til 10°C**.
4. Skriv ut resultatet for hver grad.

Eksempel på utdata:

Ved $\Delta T = 0^\circ\text{C}$ er den nye lengden: 1.00000 m

Ved $\Delta T = 1^\circ\text{C}$ er den nye lengden: 1.00001 m

Ved $\Delta T = 2^\circ\text{C}$ er den nye lengden: 1.00002 m

...

Oppgave 3 – While-løkke (simulering av trykkfall i en tank)

En trykktank lekker luft med jevnt fall på **0.5 bar per minutt**. Starttrykket er gitt av brukeren.

Lag et program som:

1. Ber brukeren skrive inn starttrykket i bar.
2. Bruk en **while-løkke** for å redusere trykket med 0.5 bar per minutt.
3. Løkken skal fortsette til trykket kommer under **1.0 bar**.
4. For hvert minutt skriver programmet ut:
5. Minutt X: Trykk = Y bar
6. Når trykket er under 1 bar, skriv:
7. Trykket er under sikkerhetsgrensen! Stopp prosessen.

Oppgave 4 – Nøstete løkker (enkelt tabellverktøy for dreiemoment)

Dreiemoment beregnes som:

$$M = F \cdot r$$

der

- F er kraft
- r er armens lengde

Lag et program som bruker **nøstete for-løkker** til å lage en tabell over dreiemoment for:

- Kraft: 10, 20, 30, ..., 100 N
- Armlengder: 0.1, 0.2, 0.3, ..., 1.0 m

Programmet skal skrive ut noe slikt:

$F = 10\text{ N}$

$r = 0.1\text{ m} \rightarrow M = 1.0\text{ Nm}$

$r = 0.2\text{ m} \rightarrow M = 2.0\text{ Nm}$

...

$F = 20\text{ N}$

$r = 0.1\text{ m} \rightarrow M = 2.0\text{ Nm}$

$r = 0.2\text{ m} \rightarrow M = 4.0\text{ Nm}$

...

Hint:

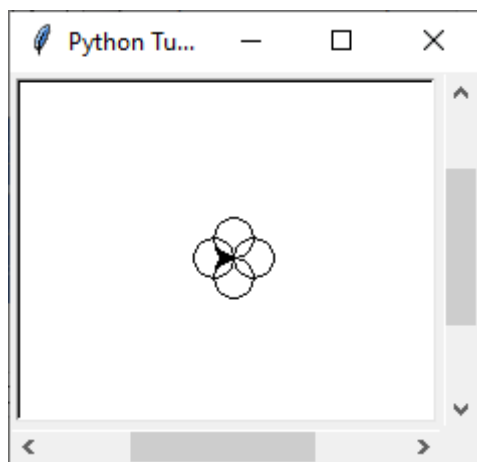
Den ytre løkken går gjennom kraftverdiene, den indre løkken går gjennom armlengdene.

Range-funksjonen godtar bare heltall som parametere. For å lage armlengdene, bruk heltall fra 1 til 10 i range-funksjonen og del dem på 10 inni løkka. En alternativ range-ekvivalent som godtar flyttall blir gjennomgått i temaet «Lister og plotting».

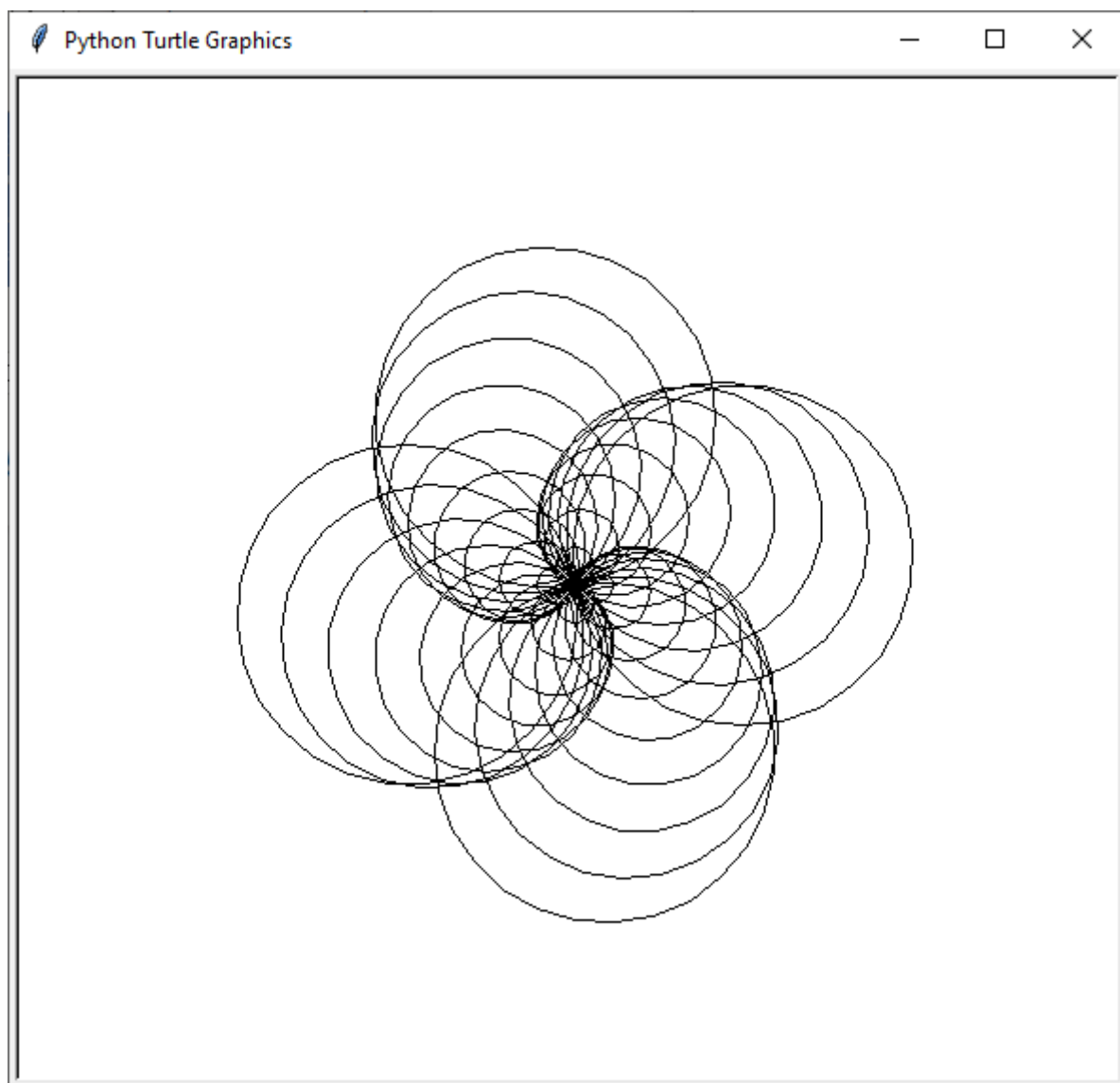
Praktisk

Denne øvingen er frivillig så du trenger ikke få den godkjent. Det vil være studentassistenter til stede i øvingstimene om du trenger hjelp eller har spørsmål rundt øvingen. Studentassistentene vil ha på seg egne t-skjorter som viser hvem de er. Gikk det greit på forkunnskapstesten trenger du ikke å gjøre denne. Synes du forkunnskapstesten var vanskelig så bør du gjøre denne øvingen. Du bør først prøve å gjøre oppgavene selv før du eventuelt spør KI om hjelp.

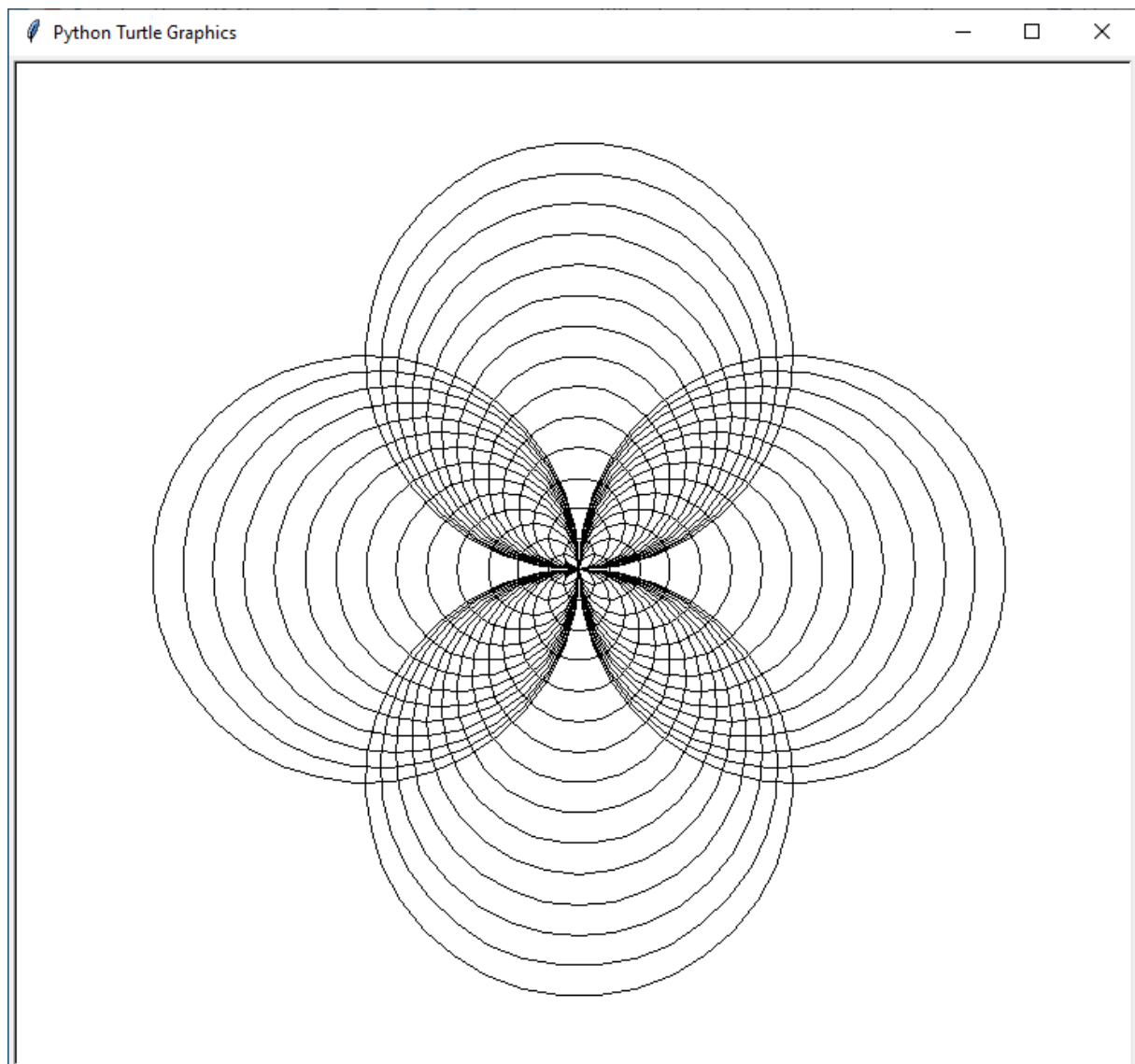
Figurer



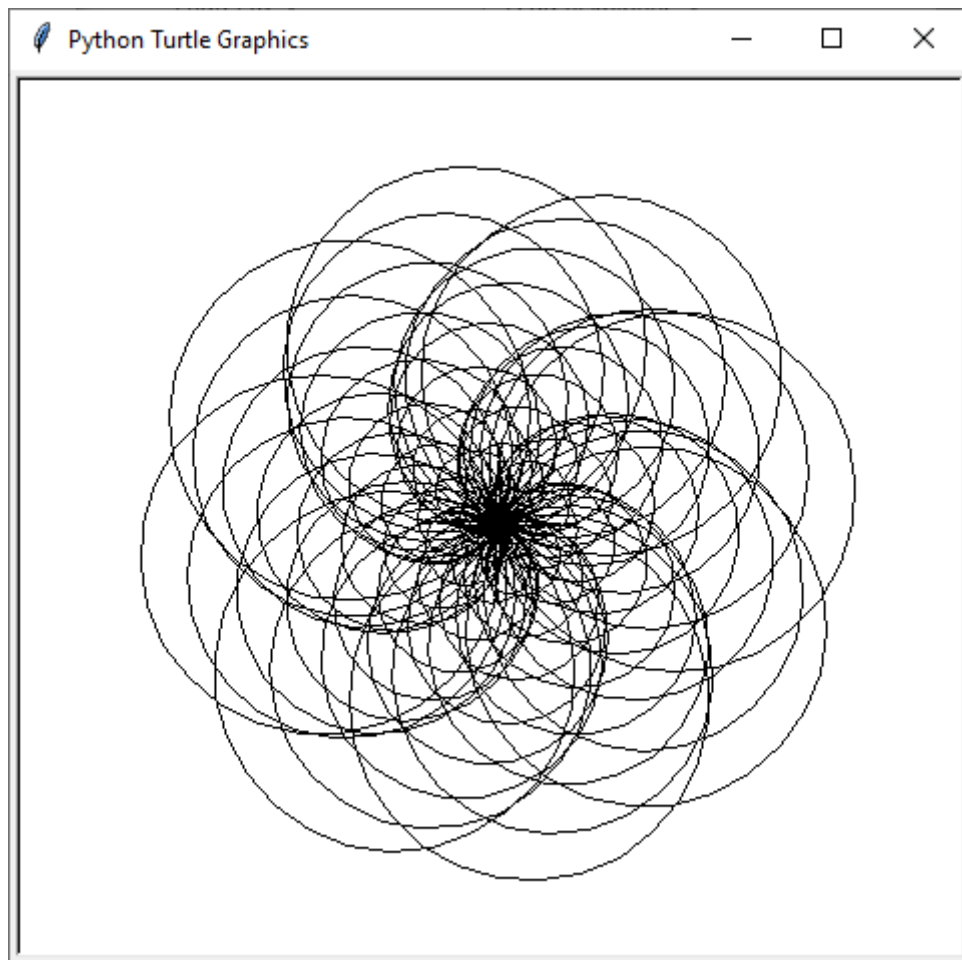
Figur 1: Fire sirkler, oppgave i



Figur 2: Eksempel på mønster, oppgave i



Figur 3: Alternativt resultat med andre parametere



Figur 4: Flere enn fire sirkler.